

App Development Project Report

App Dev Project Report

1. Student Details

Name: Patel Vansh Kalpesh

Roll Number: 25F2000889

Email: 25f2000889@ds.study.iitm.ac.in

About Me: I am a diploma level student at IIT Madras BS Degree program with a deep interest in web application development, backend systems as well as mobile applications. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: TrekFlow

Problem Statement:

You are required to build a **Trekking Management Application** web application that allows **Admin, Trek Staff, and Users (Trekkers)** to interact with the system based on their roles.

Approach:

The application was developed using Flask following a modular architecture that separates models, services, routes, validation, and utility functions. SQLAlchemy was used as the ORM for managing the SQLite database, while Jinja2 templates and Bootstrap 5 were used to build a responsive server-rendered user interface.

The system follows role-based access control with three user roles—Admin, Trek Staff, and Trekker—where each role is granted only the permissions required for its responsibilities. Session-based authentication is used for the web application, while JWT-based authentication secures the REST API. Business logic is encapsulated within service classes, and reusable decorators and validation helpers are used to keep the codebase organized and maintainable.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in styling the application, generating boilerplate code for API endpoints, and improving variable naming consistency.

The extent of AI/LLM usage is around **10-15%**, limited to **styling, API endpoints, code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Flask JWT Extended	Creating JWT tokens for API clients for their authentication
SQLite	Lightweight local database for storing user data

5. Database Schema / ER Diagram

Tables:

1. **User** — stores user profile details (id, first_name, last_name, email, password_hash, role, status)
2. **Staff Profile** — stores staff member specific fields (id, user_id, experience, phone_number, address)
3. **Trek** — stores treks and their information (id, name, location, difficulty, duration_days, available_slots, assigned_staff_id, status, start_date, end_date, description)
4. **Trek Booking** — stores user bookings for specific treks (id, user_id, trek_id, booking_date, status)

Relationships:

- One-to-One → User → Staff Profile
- One-to-Many → User → Trek Booking

- One-to-Many → Trek → Trek Booking



6. API Resource Endpoints

Endpoint	Method	Description
----------	--------	-------------

<code>/api/auth/login</code>	POST	Authenticate user and generate JWT access token
<code>/api/auth/register</code>	POST	Register user and generate JWT access token
<code>/api/treks</code>	GET	Gets all treks for admin, assigned treks for staff member, and approved treks for trekkers
<code>/api/treks</code>	POST	Add a new trek (admin only)
<code>/api/treks/{trek_id}</code>	PUT	Update any trek by its trek id (admin only)
<code>/api/treks/{trek_id}</code>	DELETE	Delete any trek by its trek id (admin only)
<code>/api/bookings</code>	GET	Gets all trek bookings for admin, assigned treks' bookings for staff member, and own bookings for trekkers
<code>/api/bookings</code>	POST	Book a trek (trekker only)
<code>/api/bookings/{booking_id}/cancel</code>	POST	Cancel any trek booking (admin only)
<code>/api/bookings/{booking_id}/restore</code>	POST	Restore any previously cancelled trek booking (admin only)
<code>/api/users</code>	GET	Get all the users (admin only)
<code>/api/users/{user_id}</code>	PUT	Update user profile (also update staff profile too, if logged in as staff member) (only self profile updating allowed)
<code>/api/users/staff/{user_id}/approve</code>	POST	Approve pending staff member request (admin only)
<code>/api/users/staff/{user_id}/reject</code>	POST	Reject pending staff member request (admin only)
<code>/api/users/{user_id}/blacklist</code>	POST	Blacklist any user (except admin) (admin only)
<code>/api/users/{user_id}/restore</code>	POST	Restore any previously blacklisted user (admin only)

YAML API Definition File:

Included separately in the submission ZIP as `api.yaml`.

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1MQhvMOwSZflmKUEFEYmmT6jXTBg15UXD/view?usp=sharing>